

的系统性测试、对 AI 交互系统如何导致故障的理解不深等。

AI 系统的一些独特性（如算法、模型）有别于传统的测试需求，需要不同的测试，例如，算法的健壮性、模型评估、模型安全等测试。基于 AI 的系统具有一些独特的质量特征，例如适应性、自主性、演化、灵活性、偏差、性能、透明度、复杂性和不确定性，这些质量特征可用作测试检查项的基础，以确定需要测试的范围和类型。

7. 小程序测试

小程序测试与 App 测试的区别包括：①获取渠道不同，小程序测试一般可以直接从微信端打开，而 App 测试需要下载安装；②测试时间长短不一样，小程序测试的开发周期和平台审核周期都比较短，而 App 测试内容较多，需要的测试时间更长。

从测试的内容及测试要点来看，涉及小程序的测试内容主要有以下几个方面：

- (1) 权限测试：包括授权登录小程序测试、能够查看的数据及用户权限等。
- (2) UI 测试：包括小程序页面展示是否准确、文字图片内容是否完善、界面的交互效果等。
- (3) 功能测试：根据小程序的各个功能模块进行测试，并对各个业务流程进行测试。
- (4) 兼容性测试：对不同操作系统测试、分享测试等。例如，目前很多小程序都只能生成分享程序图片的方式，这就需要用户做好不同屏幕的兼容适配工作。
- (5) 其他测试：例如网络测试、接口测试等。

14.4.4 其他分类

1. 回归测试

回归测试是指修改了旧代码后重新进行测试，以确认修改没有引入新的错误或导致其他代码产生错误。自动回归测试将大幅降低系统测试、维护升级等阶段的成本。

回归测试作为软件生命周期的一个组成部分，在整个软件测试过程中占有很大的比重，软件开发各个阶段都会进行多次回归测试。在渐进和快速迭代开发中，新版本的连续发布使回归测试进行得更加频繁，而在极限编程方法中，更是要求每天都进行若干次回归测试。因此，通过选择正确的回归测试策略来改进回归测试的效率和有效性是很有意义的。

回归测试需要时间、经费和人力来计划、实施和管理。为了在给定的预算和进度下，尽可能有效率和有效地进行回归测试，需要对测试用例库进行维护并依据一定的策略选择相应的回归测试包。

选择回归测试策略应该兼顾效率和有效性两个方面。常用的回归测试策略包括以下几种。

1) 再测试全部用例

选择基线测试用例库中的全部测试用例组成回归测试包，这是一种比较稳妥的测试方法，再测试全部用例具有最低的遗漏回归错误的风险，但测试成本最高。全部再测试几乎可以应用于任何情况，基本上不需要进行分析和重新开发，但是，随着开发工作的进展，测试用例不断增多，重复原先所有的测试将带来很大的工作量，往往超出了项目的预算和进度。

2) 基于风险选择测试

可以基于一定的风险标准来从基线测试用例库中选择回归测试包。首先运行最重要的、关

键的和可疑的测试，而跳过那些非关键的、优先级别低的或者高稳定的测试用例，这些用例即便可能测试到缺陷，其严重性也仅有三级或四级。一般而言，测试顺序是从主要特征到次要特征。

3) 基于操作剖面选择测试

如果基线测试用例库的测试用例是基于软件操作剖面开发的，则测试用例的分布情况反映了系统的实际使用情况。回归测试所使用的测试用例个数可以由测试预算确定，回归测试可以优先选择那些针对最重要或最频繁使用功能的测试用例，释放和缓解最高级别的风险，有助于尽早发现那些对可靠性有最大影响的故障。这种方法可以在一个给定的预算下最有效地提高系统可靠性，但实施起来有一定的难度。

4) 再测试修改的部分

当测试者对修改的局部化有足够的信心时，可以通过相依性分析识别软件的修改情况并分析修改产生的影响，将回归测试局限于被改变的模块和它的接口上。通常，一个回归错误一定涉及一个新的、修改的或删除的代码段。在允许的条件下，回归测试应尽可能地覆盖受到影响的部分。

再测试全部用例的策略是最安全的策略，但已经运行过许多次的回归测试不太可能揭示新的错误，而且很多时候，由于时间、人员、设备和经费的原因，不允许选择再测试全部用例的回归测试策略，此时，可以选择适当的策略进行缩减的回归测试。

在实际工作中，回归测试需要反复进行，当测试者一次又一次地完成相同的测试时，这些回归测试将变得非常令人厌烦，而在大多数回归测试需要手工完成的时候尤其如此，因此，需要通过自动测试来实现重复的和一致的回归测试。通过测试自动化可以提高回归测试效率。为了支持多种回归测试策略，自动测试工具应该是通用和灵活的，以便满足达到不同回归测试目标的要求。例如，安排新的测试者完成手工回归测试，分配更有经验的测试者开发新的测试用例，编写和调试自动测试脚本，做一些探索性的或 Ad-Hoc 测试（随机测试）。还可以在不影响测试目标的情况下，鼓励测试者创造性地执行测试用例，变化的输入、按键和配置有助于激励测试者揭示新的错误。

2. 冒烟测试

冒烟测试这一术语源自硬件行业，它是对一个硬件或硬件组件进行更改或修复后，直接给设备加电，如果没有冒烟，则该组件就通过了测试。在软件中，冒烟测试这一术语描述的是在将代码更改嵌入产品的源树之前对这些更改进行验证的过程。在检查了代码后，冒烟测试是确定和修复软件缺陷的最经济有效的方法。冒烟测试设计用于确认代码中的更改会按预期运行，且不会破坏整个版本的稳定性。

冒烟测试的对象是每一个新编译的需要正式测试的软件版本。通过冒烟测试，在软件代码正式编译并交付测试之前，先尽量消除其表面的错误，减少后期测试的负担。冒烟测试的执行者是版本编译人员。因此可以说，冒烟测试是预测试。在实际的软件测试工作中，冒烟测试在软件研发的不同阶段有所不同，大体可以分为三类：

(1) 形成集成测试版本以前：验证各个单元能够成功执行，并保证测试版本能够顺利集成。

(2) 形成集成测试版本：以保证新的或者更改过的代码不破坏集成版本的完成性和稳定性。

(3) 后期预测测试缺陷的修正：针对每个缺陷所做的缺陷修正都要先在干净的链接环境中进行冒烟测试，测试通过后才能更新相关软件版本。

冒烟测试在软件生命周期中所占的时间比例较低，同时具有注重通过性、轻细节的特点，因此经常被开发、测试人员所忽视。事实上，冒烟测试是软件测试过程中一个不可或缺的节点，一个好的冒烟测试过程，对于软件测试效率的提升具有重要意义。

14.5 软件测试的组织

14.5.1 测试过程

软件测试过程主要包括以下几个阶段：

(1) 项目需求分析阶段：项目需求是整个测试过程的前置条件，也是任何一个测试工作开始前的第一步。所谓项目需求分析，是指测试员分析和确定被测试的项目的测试工作安排，也就是该项目需要测哪些模块，什么样的结果是符合产品需求的预期结果。

(2) 制订测试计划阶段：测试计划制订阶段的主要任务就是编写完整的测试流程安排，测试哪些模块，用什么方法测试（例如，常见的冒烟测试、随机测试、黑盒测试、性能测试等），以及不同测试阶段的测试人员的工作安排、工作目标等。这个阶段对测试主管的整体测试调控能力有很高的要求。

(3) 设计测试文档阶段：这个阶段是整个软件测试的基础，也是核心阶段。这个阶段的主要工作是参考项目的设计原型图、产品的设计需求和详细设计标准等编写测试用例。

(4) 搭建测试环境阶段：不同的产品对测试环境有着不同的要求。能做好软件测试工程师这门工作的一个指标就是能独立搭建项目的测试环境。

(5) 执行测试阶段：这个阶段就开始正式的测试工作了。一般包括单元测试、集成测试、系统测试、验收测试以及回归测试等。每个阶段对应了不同的测试具体执行细节。

(6) 缺陷管理：缺陷管理主要是用于收集、管理和沟通项目缺陷的过程，重点考察的是缺陷的识别和缺陷的解决。这个阶段一般会用到一些系统化的缺陷管理工具。

14.5.2 测试管理

1. 配置管理

软件测试配置管理一般应用过程方法和系统方法来建立软件测试管理体系，也就是把软件测试管理作为一个系统，对组成这个系统的各个过程加以识别和管理，以实现设定的系统目标。同时要使这些过程协同作用、互相促进，从而使它们的总体作用大于各个过程之和。软件测试配置管理的主要目标是在设定的条件限制下，尽可能发现和找出软件缺陷。测试配置管理是软件配置管理的子集，作用于测试的各个阶段。其管理对象包括软件测试计划、测试方案（用例）、测试版本、测试工具及环境、测试结果等。

配置管理的目标：

- (1) 控制和审计测试活动的变更；
- (2) 在测试项目的里程碑建立相应的基线；
- (3) 记录和跟踪测试活动变更请求；
- (4) 相应的软件测试活动或产品（work products）被标识、控制，并且是可用的。

配置管理的承诺：

- (1) 每个测试项目的配置管理责任明确；
- (2) 配置管理贯穿项目的整个测试活动；
- (3) 配置管理应用于所有的测试配置项，包括支持工具；
- (4) 建立配置库和基线库（baseline）；
- (5) 定期评审基线库内容和测试配置项活动。

2. 过程管理

软件测试的过程管理并不仅限于测试阶段，所有的测试都是基于需求的，在需求分析阶段，测试团队要依据需求文档提取测试点。

测试的过程管理的基本内容如下：

(1) 测试项目启动阶段：首先需要确定项目负责人，即项目小组组长，项目组长确定以后，才可以组建整个测试小组，配合开发等部门开展工作；其次要参加有关项目计划、分析和设计的会议，获得必要的需求分析、系统设计文档，以及相关产品 / 技术知识的培训和转移。

(2) 测试计划阶段：确定测试范围、测试策略和方法，对风险、日程表、资源等进行分析和估计。

(3) 测试设计阶段：制定测试的技术方案，设计测试用例，选择测试工具，编写测试脚本等。测试用例设计需要做好各项准备再开始进行，最后还需要其他部门帮忙评审测试用例。

(4) 测试执行阶段：搭建相关的测试环境，准备测试数据，执行测试用例，对发现的软件缺陷进行报告、分析、跟踪等。测试执行不涉及较高的技术性，但它是测试的基础，直接关系到测试的可靠性、客观性和准确性。

(5) 测试结果的审查和分析阶段：测试执行结束后，需要对测试结果进行整体或综合的分析，以确定软件产品质量的当前状态，为产品的改进或发布提供数据和依据。从管理上讲，需要组织好测试结果的评审和分析会议，做好测试报告或质量报告的编写和评审。

3. 缺陷管理

软件缺陷有很多，从不同的角度可以将缺陷分为不同的种类。

- (1) 按照测试种类可以将软件缺陷分为界面类、功能类、性能类、安全性类、兼容性类等。
- (2) 按照缺陷的严重程度可以将缺陷划分为严重、一般、次要、建议。
- (3) 按照缺陷的优先级不同可以将缺陷划分为立即解决、高优先级、正常排队、低优先级。
- (4) 按照缺陷的发生阶段不同可以将缺陷划分为需求阶段缺陷、构架阶段缺陷、设计阶段缺陷、编码阶段缺陷、测试阶段缺陷。

当测试人员发现软件中存在的缺陷之后，我们还需要对这些缺陷进行统一管理。

- (1) 确定缺陷优先级：在评估缺陷的优先级时，可以从单个或多个维度评估，通常情况下